

SMoL-X (Small Language with eXtensions)

Define the Grammar Definition	2
Backus-Naur Form (BNF).....	2
Example Programing Style	2
1.1 Abstract Syntax Tree	3
1.2 Parser Implementation	4
1.3 Substitution Function	9
1.4 Evaluator Implementation.....	11
1.5 Instrumentation.....	12
1.6 Testing Requirements	14
2.1 Environment Data Structure	15
2.2 Environment-Based Evaluator	16
2.3 Instrumentation for Environments	17
2.4 Preserving Static Scoping.....	18
Duplication Stress Test	19
Shadowing Verification Test.....	21
Deep Nesting Stress Test.....	22
Written Report	23

Define the Grammar Definition

Backus-Naur Form (BNF)

;; Trying to match the example programming style

;; Recursively evaluating the expression

<expr> ::= <number>

| <boolean>

| <variable>

| (+ <expr> <expr>)

| (if <expr> <expr> <expr>)

| (let1 (<variable> <expr>) <expr>)

;; Basic building blocks

<number> ::= integer literals

<boolean> ::= #F | #T

<variable> ::= identifier (symbol)

Example Programing Style

;; What our language will look like

;; Simple arithmetic

(2 + 3) ;; => 5

;; Conditionals if else

(if true 10 then 20) ;; => 10

(if false 10 then 20) ;; => 10

;; Local binding

(let (x 5) (x + 3)) ;;=> 8

;; Nested bindings

(let (x 10)

(let (y 20)

(x + y))) ;;=> 30 ;

;; Shadowing

(let (x 5)

(let (x 10)

x)) ;; => 10

1.1 Abstract Syntax Tree

Define your AST using (define-type). Your AST should include constructors for:

Numbers (numE)

Booleans (boolE)

Variables (varE)

Addition (plusE)

Conditionals (ifE)

Local binding (let1E)

```
#lang plait

;; =====
;; 1. AST Definition
;; =====
(define-type Exp
  [numE (n : Number)]
  [boolE (b : Boolean)]
  [varE (name : Symbol)]
  [plusE (left : Exp) (right : Exp)]
  [ifE (test : Exp) (then-part : Exp) (else-part : Exp)]
  [let1E (var : Symbol) (val : Exp) (body : Exp)])

;; =====
;; 2. Value Definition
;; (The result of an evaluation)
;; =====
(define-type Value
  [numV (n : Number)]
  [boolV (b : Boolean)])
```

```
:: =====  
:: 3. Environment Definition  
:: (Needed for the Environment-based interpreter)  
:: =====  
(define-type Env  
  [mtEnv] ;; Empty environment  
  [extendEnv (name : Symbol) (value : Value) (rest : Env)])
```

1.2 Parser Implementation

Your parser must convert s-expressions into AST nodes. Use the following helper functions

(s-exp-number?) - Check if s-expression is a number

(s-exp-symbol?) - Check if s-expression is a symbol

(s-exp-list?) - Check if s-expression is a list

(s-exp>number) - Extract number from s-expression

(s-exp>symbol) - Extract symbol from s-expression

(s-exp>list) - Extract list from s-expression

```
;; =====  
  
;; PART 2: PARSER  
  
;; =====  
  
{define (num [string : S-Exp]) : Exp  
  (numE (s-exp->number string))}  
  
{define (var [string : S-Exp]) : Exp  
  (varE (s-exp->symbol string))}  
  
{define (plus [string : S-Exp]) : Exp  
  (let* ([parts (s-exp->list string)])
```

```
(let* ([left (parse (first parts))]

      [right (parse (third parts))])

  (cond

    [(and (numE? left) (numE? right))

     (numE (+ (numE-n left) (numE-n right)))]

    [else

     (plusE left right)]))}
```

```
{define (iff [string : S-Exp]) : Exp

  (let* ([parts (s-exp->list string)])

    (let* ([test (parse (second parts))]

          [then (parse (third parts))]

          [elsse (parse (list-ref parts 4))])

      (ifE test then elsse))))}
```

```
{define (let [string : S-Exp]) : Exp

  (let* ([parts (s-exp->list string)]

        (let* ([var (s-exp->symbol (second parts))]

              [val (parse (fourth parts))])
```

```
[body (parse (list-ref parts 4))])
```

```
(let1E var val body))))}
```

```
{define (parse [s : S-Exp]) : Exp
```

```
(cond
```

```
[(s-exp-match? `(if ANY ANY then ANY) s)(iff s)]
```

```
[(s-exp-match? `(let SYMBOL = ANY ANY) s)(let s)]
```

```
[(equal? `true s) (boolE #t)]
```

```
[(equal? `false s) (boolE #f)]
```

```
[(s-exp-match? `NUMBER s) (num s)]
```

```
[(s-exp-match? `SYMBOL s) (var s)]
```

```
[(s-exp-match? `(ANY + ANY) s)(plus s)]
```

```
[else (error 'parse "Unknown Expression")]
```

```
))
```

```
(test (parse `(let a = 10 (1 + a))) (let1E 'a (numE 10) (plusE (numE 1) (varE 'a))))
```

```
{test (parse `(if x 10 then 20) ) (ifE (varE 'x) (numE 10) (numE 20))}
```

```
{test (parse `1) (numE 1)}
```

```
{test (parse `(1 + (1 + 2))) (numE 4)}
```

```
{test (parse `(1 + (x + 2))) (plusE (numE 1) (plusE (varE 'x) (numE 2))))}
```

```
{test (parse `(1 + y)) (plusE (numE 1) (varE 'y)))}
```

```
{test (parse `(y + 2)) (plusE (varE 'y) (numE 2)))}
```

```
{test (parse `true) (boolE #t)}
```

```
{test (parse `false) (boolE #f)}
```

1.3 Substitution Function

The substitution function is the heart of your interpreter. It must: Replace all free occurrences of a variable with a value Preserve static scoping NOT substitute into shadowed variables Handle all expression types correctly

```
:: =====  
  
:: PART 3: SUBSTITUTION INTERPRETER  
  
:: =====  
  
;; 1. Setup the Substitution Counter  
(define sub-count (box 0))  
  
(define (reset-sub-counter!)  
  (set-box! sub-count 0))  
  
(define (get-sub-count)  
  (unbox sub-count))  
  
;; 2. The Substitution Function  
;; Critical requirement: Must correctly handle shadowing  
(define (subst [var : Symbol] [val : Exp] [expr : Exp]) : Exp  
  (begin  
    (set-box! sub-count (+ 1 (unbox sub-count))) ;; Increment counter  
    (type-case Exp expr  
      [(numE n) expr]  
      [(boolE b) expr]  
      [(varE x) (if (symbol=? x var) val expr)]  
      [(plusE l r) (plusE (subst var val l) (subst var val r))]  
      [(ifE test then elsse)  
        (ifE (subst var val test) (subst var val then) (subst var val elsse))]  
      [(let1E bound-var bound-body let-body)  
        (if (symbol=? bound-var var)
```

```

;; Shadowing: If names match, do NOT substitute into the body
(let1E bound-var (subst var val bound-body) let-body)

;; No shadowing: Substitute into both parts
(let1E bound-var (subst var val bound-body) (subst var val let-body))))))

;; 3. The Interpreter
;; Rewritten to avoid 'let' identifier binding issues
(define (interp [expr : Exp]) : Exp
  (type-case Exp expr
    [(numE n) expr]
    [(boolE b) expr]
    [(varE x) (error 'interp (string-append "Free variable: " (symbol->string x)))]

    [(plusE l r)
     ;; We extract the numbers directly to avoid 'v1' unbound errors
     (numE (+ (numE-n (interp l))
              (numE-n (interp r))))]

    [(ifE test then else)
     ;; Use type-case on the result of the test directly
     (type-case Exp (interp test)
       [(boolE b) (if b (interp then) (interp else))]
       [else (error 'interp "If test must be a boolean")])]

    [(let1E var val body)
     ;; Substitute the evaluated value into the body, then interp the result
     (interp (subst var (interp val) body)))]

    _ (error 'interp "Invalid expression")

  )

;; --- Verification Tests ---
;;(test (interp (parse `(let x = 5 (let x = 10 x)))) (numE 10))
;;(test (interp (parse `(let x = 5 (x + 2)))) (numE 7))
;;(test (interp (parse `(if true 1 then 2))) (numE 1))

```

1.4 Evaluator Implementation

Implement an evaluator using substitution:

1.5 Instrumentation

Add a counter to track substitution operations:

1.6 Testing Requirements

Include at least 15 tests covering:

Simple arithmetic (3 tests)

Conditionals with true/false (3 tests)

Variable binding and lookup (3 tests)

Nested bindings (3 tests)

Shadowing behavior (3 tests)

2.1 Environment Data Structure

Define the environment type and basic operations:

2.2 Environment-Based Evaluator

Implement the evaluator that uses environments instead of substitution:

2.3 Instrumentation for Environments

Add a lookup counter:

2.4 Preserving Static Scoping

Critical verification: Your environment-based evaluator must produce the same results as your substitution evaluator for all valid programs. This is a fundamental requirement

Duplication Stress Test

Purpose: Demonstrate repeated evaluation under substitution.

Code:

Measurements:

Substitution count (for substitution interpreter)

Lookup count (for environment interpreter)

Execution time (optional, but recommended)

Result verification (both must produce the same answer)

Shadowing Verification Test

Purpose: Confirm static scope is preserved in both interpreters. Create a program with multiple levels of shadowing to verify correctness.

Code:

Measurements:

Substitution count (for substitution interpreter)

Lookup count (for environment interpreter)

Execution time (optional, but recommended)

Result verification (both must produce the same answer)

Deep Nesting Stress Test

Purpose: Stress the interpreter with deeply nested let-bindings. Create a program with 10-20 levels of nesting to measure performance differences

Code:

Measurements:

Substitution count (for substitution interpreter)

Lookup count (for environment interpreter)

Execution time (optional, but recommended)

Result verification (both must produce the same answer)

Written Report

Report Structure (3-5 pages)

Expected Analysis:

Your report should explain:

Why the substitution interpreter performs more operations Where the duplication occurs
in substitution How environments eliminate redundant work When environments provide
the most benefit

Language Definition and Grammar (0.5-1 page)

Present your SMoL-X grammar in BNF notation

Explain each construct briefly

Provide 2-3 example programs with expected output

Describe any design choices you made

Substitution Interpreter (1-1.5 pages)

Explain the substitution algorithm

Describe how you handle shadowing

Show key code snippets (AST definition, substitution function)

Discuss challenges encountered and how you solved them

Environment Interpreter (1-1.5 pages)

Explain the environment-based approach

Describe your environment data structure

Show key code snippets (lookup, extend, eval-env)

Explain how it differs from substitution

Experimental Setup and Results (1-1.5 pages)

For each stress test, include:

Description of the test program

Substitution count

Lookup count

Analysis of the results

Comparison Table:

Test	Substitution Ops	Lookup Ops	Speedup Factor
------	------------------	------------	----------------

Duplication			
Shadowing			
Deep Nesting			

Analysis and Conclusions (0.5-1 page)

Must address:

1. Why substitution can be inefficient:

Explain the linear time complexity per binding

Describe how repeated variable use causes duplication

Give specific examples from your stress tests

2. How caching substitution (environments) improves performance:

Explain the space-time tradeoff

Describe constant-time lookup vs. linear-time substitution

Quantify the improvement from your experiments

3. Why environment-based evaluation is closer to real implementations:

Discuss practical considerations (memory, speed)

Explain why modern interpreters use environments

Connect to what you learned about language implementation

4. Limitations of your implementation:

What features would a real language need?

What optimizations could be added?

What did you learn from this project?